



Project name: SCENIX

3D Scene Reconstruction from 360° Images
Individual Internship Progress Report — Summer 2026



Duration: 1 June 2026 – 14 July 2026

Repository: <https://github.com/deep-1005/Scenix>

Student name: Dhriti Kamani [PES1UG24AM085]

Guide Professor: Dr. Adithya Balasubramanyam

1. Project Overview

The SCENIX project builds an end-to-end platform that converts 360° panoramic captures of a physical scene into an interactive, measurable 3D reconstruction. A scene is photographed with an Insta360 camera, the equirectangular panoramas are converted into pinhole-camera cubemap faces, camera poses and a sparse point cloud are recovered with COLMAP, a photorealistic Gaussian splat is trained with FastGS, and the result is served through a web application that also targets object (evidence) detection, classification, measurement and automated report generation. The intended application is forensic documentation of crime scenes, where the reconstruction must be both visually faithful and defensible.

The pipeline implemented over the internship is:

360° capture → CubeMap generation → COLMAP sparse reconstruction → cleaning (RANSAC/DBSCAN) → FastGS Gaussian splatting → SuperSplat/point-cloud inspection → (detection, classification, measurement and reporting)

All orchestrated behind a web interface with per-stage progress tracking. The work was divided equally among three team members, each owning one workstream end-to-end while collaborating on integration and testing. This report covers my individual contribution.

2. My Role and Responsibilities

My contribution to the SCENIX project was developing the software platform that transforms the research pipeline into a usable product. I designed and implemented the web application that automates the complete reconstruction workflow—from cubemap generation, COLMAP reconstruction, point cloud cleaning using **RANSAC and DBSCAN**, **FastGS-based Gaussian Splatting**, and visualization—into a single-click pipeline. My work included integrating and orchestrating the backend pipeline, resolving compatibility issues between different reconstruction stages, and developing an **interactive UI** with real-time job tracking, pipeline status monitoring, point cloud and Gaussian Splat visualizations, and scene management. I also worked on deploying the application on the ISFCR system and configuring **LAN-based** access so reconstructions could be accessed and monitored from multiple machines. This workstream transformed the team's research workflow into a practical end-to-end 3D reconstruction product. In parallel, I contributed to the 360° reconstruction workflow by capturing **360° videos**, extracting and preprocessing frames through blur filtering and removal of capture operators, and preparing datasets for automated reconstruction. I also evaluated and integrated reconstruction tools such as COLMAP, FastGS, SuperSplat, and **SuGaR**, experimenting with different reconstruction parameters and pipeline configurations to improve the quality of the generated Gaussian Splats.

3. Work Carried Out

Phase 1 — Learning the domain and first test captures

The internship opened with self-study rather than code: working through introductory material on 3D Gaussian splatting and photogrammetry, and installing the core tools we expected to need — Blender, Unity and COLMAP. We validated our understanding against a public dataset (the COLMAP project's South Building set) before trusting our own captures, running it through COLMAP and inspecting results in MeshLab and Brush.

We then took our first real captures: a bottle photographed on a phone, and a spiral-bound book photographed with the Insta360 360° camera — deliberately simple, textured objects that would reveal whether our reconstruction setup worked at all before we pointed it at a room. The book capture in particular became a recurring test case over the following days as we compared tools: COLMAP, RealityScan, and Brush + SuperSplat all produced a reconstruction from it, with RealityScan initially standing out for speed and for exporting a usable .obj into Unity, where we built a first test scene.



Figure 1 — Test Capture: bottle photographed with phone camera for first reconstruction trials

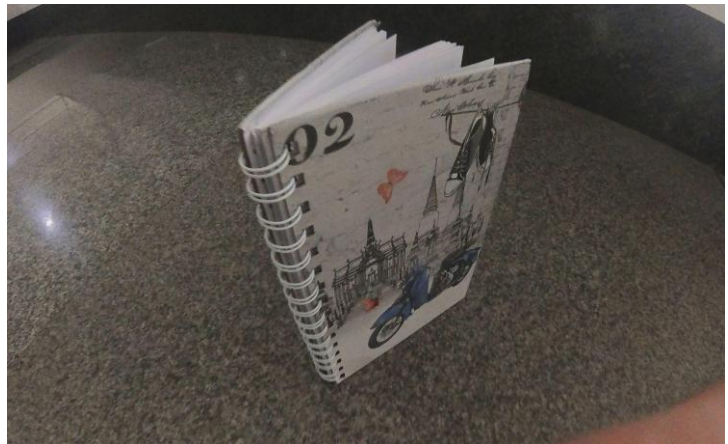


Figure 2 — Test Capture – book photographed with the Insta360 360 degree camera, used to compare the COLMAP, RealityScan and Brush

Phase 2 — Processing 360° room captures and reconstruction pipeline

After understanding the reconstruction workflow, I shifted to processing full-room captures using the Insta360 360° camera. I investigated methods for converting 360° equirectangular images into perspective and cubemap views using tools such as Hugin and custom Python scripts, enabling them to be processed by photogrammetry software. I carried out reconstruction experiments with COLMAP, FastGS, Nerfstudio and SuperSplat across multiple systems using datasets ranging from a few hundred to over 5,000 images, while also exploring tools such as LixelStudio. These experiments helped establish a working pipeline for generating point clouds and Gaussian Splatting outputs from 360° room captures.

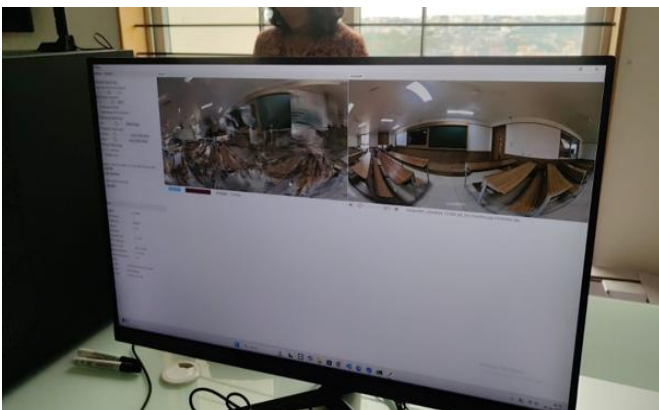


Figure 3 – Equirectangular 360° room images captured using the Insta360 camera

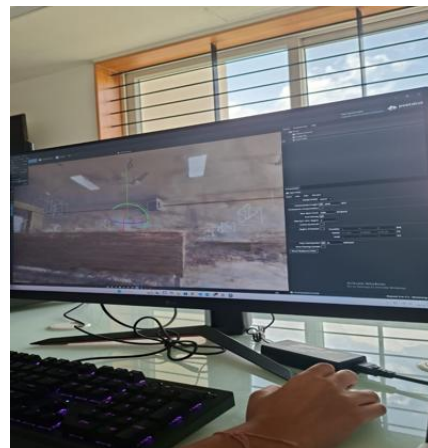


Figure 4 – Reconstruction workflow using Brush, showing direct processing of 360° images to generate an initial scene.

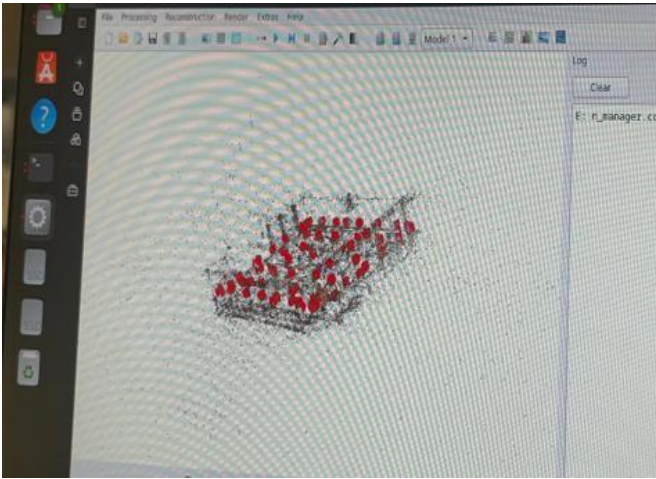


Figure 5 – Sparse reconstruction generated by COLMAP from the converted perspective image dataset, illustrating estimated camera poses and sparse point cloud.

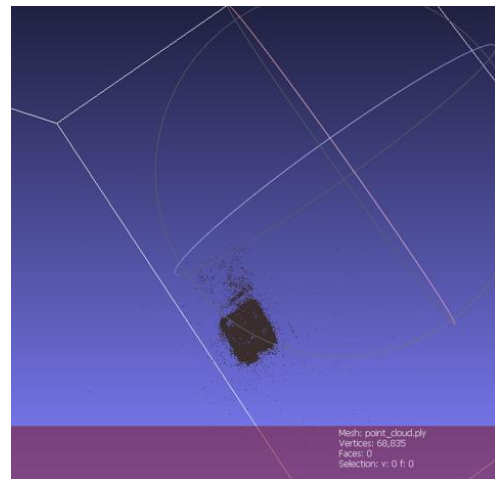


Figure 6 – MeshLab visualization of the reconstructed point_cloud.ply generated from the COLMAP output.

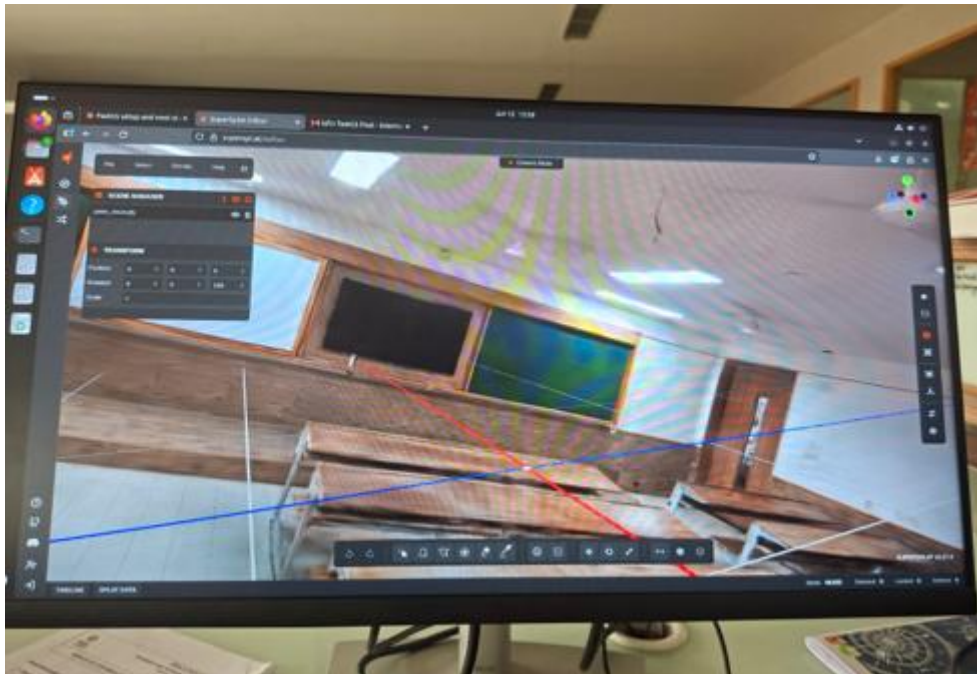


Figure 7 – Gaussian Splatting visualization of the reconstructed classroom scene using SuperSplat, generated from the processed point cloud.

Phase 3 — Full pipeline automation and application development

With the reconstruction pipeline established, I focused on automating the complete workflow and developing the accompanying application. I integrated **the user interface** into the pipeline so the entire chain—cubemap generation, COLMAP reconstruction, point cloud cleaning, Gaussian Splatting training (FastGS), post-processing, and the downstream detection, classification, measurement and reporting stages—could run as a single tracked job. The interface was rebuilt around this workflow, with the scene list displaying per-scene progress bars and live status indicators (running, queued, completed and failed), together with a horizontal pipeline tracker showing the current stage of processing. Dedicated tabs were added for the cubemap gallery (including perspective-view previews and download support), an interactive Three.js viewer for the COLMAP point cloud, a Gaussian Splat preview, and a full interactive Gaussian Splat viewer using the [@mkkellogg/gaussian-splats-3d](https://github.com/mkkellogg/gaussian-splats-3d) library.

To improve the quality of the reconstruction pipeline, I integrated **RANSAC and DBSCAN** (figure 10) immediately after the COLMAP stage to automatically clean the generated sparse point cloud before it was passed to FastGS. RANSAC was used to identify and remove dominant planar structures and outlier points, while DBSCAN clustered the remaining points to eliminate isolated noise and retain only the meaningful

scene geometry. This preprocessing step produced a cleaner and more reliable point cloud, resulting in better Gaussian Splat initialization and noticeably improved reconstruction quality compared to using the raw COLMAP output directly.

The application also included controls to stop and resume long-running stages such as cubemap generation, COLMAP and Gaussian training without restarting the complete pipeline, together with a searchable scene manager for handling multiple reconstructions. After the automated image-based workflow was completed, I looked into the pipeline to support 360° video inputs by extracting frames from captured videos, removing frames containing the operators, and automatically feeding the resulting images into the same reconstruction pipeline.

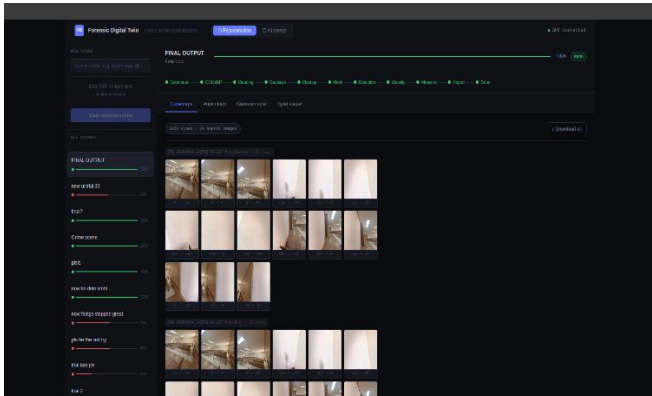


Figure 7 — Cubemap Generation on our website

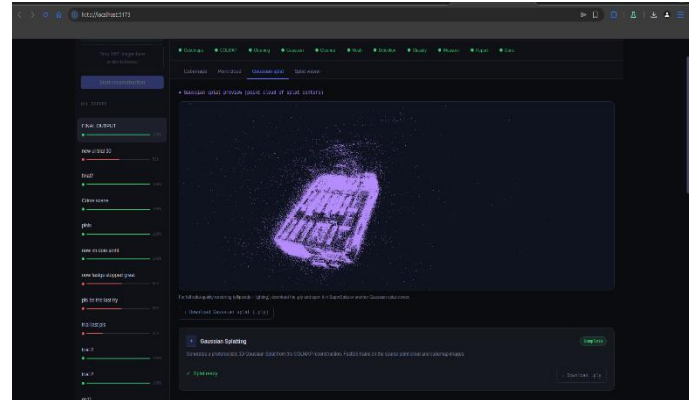


Figure 8 — Pipeline UI: stage tracker

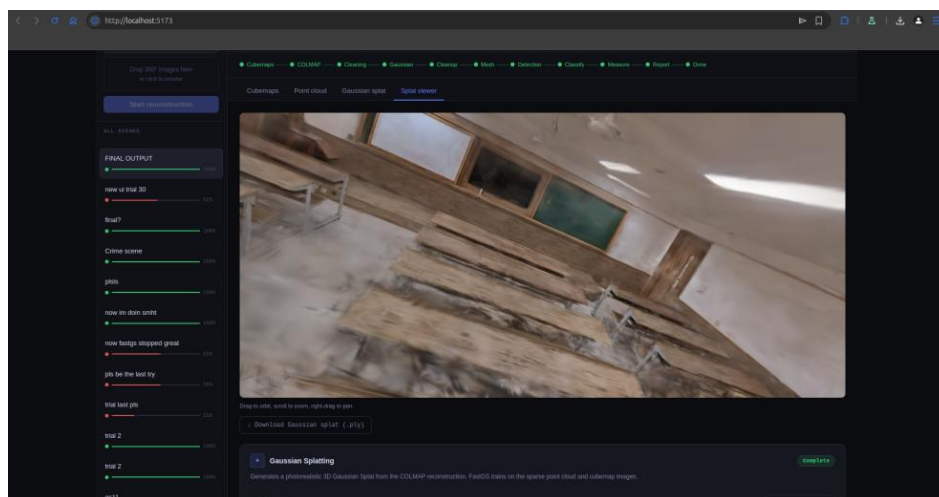


Figure 9 — Gaussian splat preview (splat centers) rendered in the browser with .ply download.

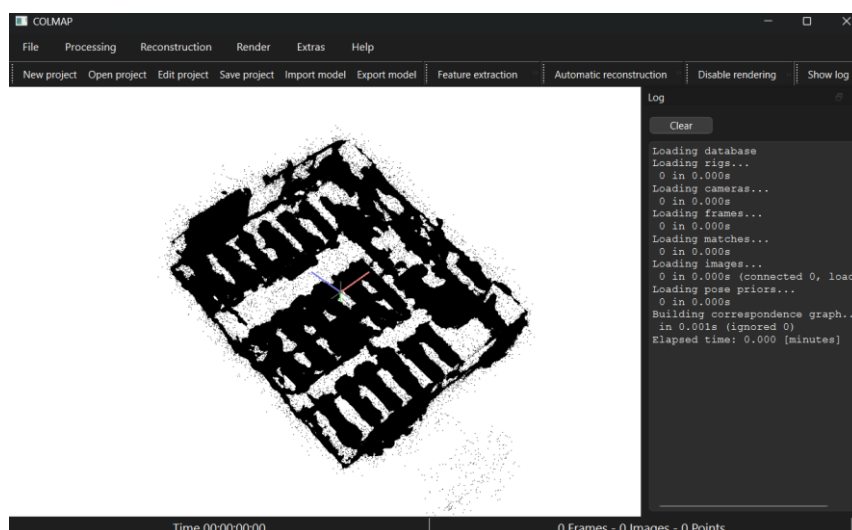


Figure 10 — Ransac and dbscan output for room after point cloud generation

Phase 3 — Deployment and remote access

I worked on making the platform reachable beyond the host machine: running the server on the ISFCR PC with the frontend on another PC over LAN (nginx), establishing SSH remote access (ssh cave@10.1.12.38), and exposing the site externally with ngrok — which worked but hit free-tier bandwidth limits when streaming splat data — leading to duckdns.org subdomains and IP configuration as the recommended approach. Docker was attempted for portability, and I researched cloud infrastructure options (MeitY-empanelled providers, AWS with GPU, Azure student plan, DigitalOcean) for the production-scale path.

Phase 4 — Optimizing the FastGS output

The final phase focused on improving the quality of the Gaussian Splat reconstruction beyond the initial FastGS output. Although the generated reconstruction successfully captured the overall structure of the scene, it contained floaters, blurred regions, and missing geometry because only 585 of the 1,050 perspective images were successfully registered by COLMAP. Since FastGS directly depends on the quality of the sparse reconstruction generated by COLMAP, incomplete camera registration propagated through the remainder of the pipeline and resulted in degraded Gaussian Splat quality.

To improve the reconstruction, I first evaluated different FastGS configurations and integrated **SuGaR (Surface-Oriented Gaussian Reconstruction)** as a post-processing stage. However, SuGaR relies on an already accurate sparse reconstruction to generate a surface representation. Since the underlying COLMAP model itself was fragmented, SuGaR inherited the same errors and amplified the existing artifacts rather than correcting them. This demonstrated that improving downstream reconstruction algorithms alone would not solve the problem; instead, the quality of the upstream reconstruction pipeline had to be improved.

The focus therefore shifted towards optimizing the COLMAP stage. I experimented with different camera models by replacing SIMPLE_PINHOLE with SIMPLE_RADIAL, evaluated different feature matching strategies by replacing sequential matching with exhaustive matching, introduced an **image_undistorter** stage to generate PINHOLE camera models compatible with FastGS, and trained FastGS at half resolution to overcome GPU memory limitations during long training runs.

Alongside these optimizations, I investigated a **dynamic cubemap generation strategy** as a potential improvement to the preprocessing pipeline. The current workflow converts each 360° panorama into a fixed number of perspective images before reconstruction. However, COLMAP fundamentally depends on sufficient overlap between neighbouring images to establish feature correspondences and recover reliable camera poses. My proposed approach is to make cubemap generation **adaptive rather than fixed**, using feedback from the COLMAP reconstruction to determine where additional perspective views should be generated. Regions with poor image registration or insufficient overlap could automatically receive more perspective views at intermediate viewing angles, increasing feature matches and improving camera registration. This would produce a denser and more complete sparse point cloud before Gaussian Splat training, ultimately leading to higher-quality reconstructions while generating additional images only where they are needed instead of uniformly across the entire dataset.

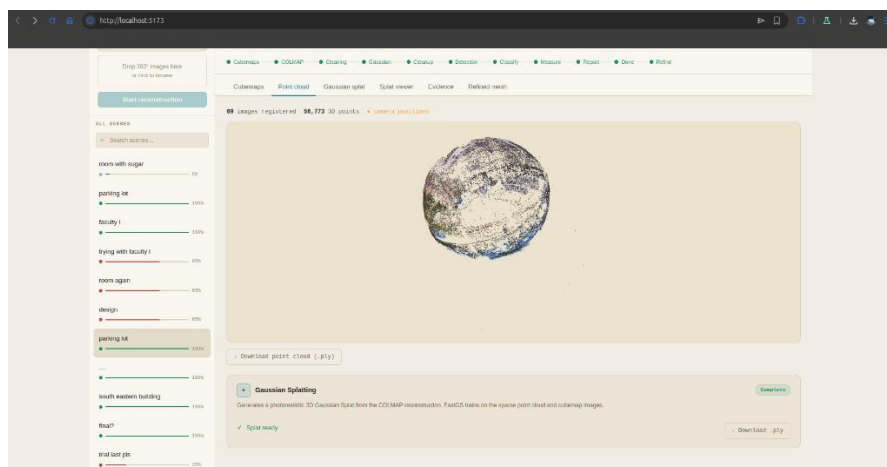


Figure 11 - Point cloud of Parking lot dataset with the final UI

Figure 12 – Terminal Running in the background

4. Key Outcomes

- Developed a fully functional end-to-end web platform that automates the complete 3D reconstruction workflow—from 360° image and video input to interactive Gaussian Splat visualization—through a single interface with real-time pipeline progress tracking, scene management, and visualization of intermediate reconstruction outputs.
- Successfully integrated the complete reconstruction pipeline, including cubemap generation, COLMAP, RANSAC, DBSCAN, FastGS, SuperSplat, and visualization modules, into a single automated workflow. The addition of point cloud preprocessing significantly improved the quality and reliability of the Gaussian Splat reconstruction.
- Implemented support for 360° video-based reconstruction by developing a preprocessing workflow for frame extraction, blur filtering, removal of capture operators, and automatic integration of the processed frames into the existing reconstruction pipeline.
- Deployed the application on the ISFCR workstation with LAN-based accessibility, investigated remote deployment using SSH, ngrok, and DuckDNS, and evaluated cloud infrastructure options to support future large-scale deployment.
- Evaluated and optimized multiple Gaussian Splatting approaches, including FastGS and SuGaR, while experimenting with different COLMAP configurations, camera models, and matching strategies to improve reconstruction accuracy and overall output quality.

5. Challenges and Learnings

- Processing 360° imagery required considerable experimentation to determine an effective workflow for converting equirectangular panoramas into perspective images that could be reliably processed by conventional photogrammetry software such as COLMAP.
- Integrating multiple independent tools—including COLMAP, FastGS, SuperSplat, RANSAC, DBSCAN, and the web application—into a single automated pipeline presented significant compatibility and orchestration challenges, requiring careful handling of intermediate outputs, formats, and execution order.
- Reconstruction quality was found to depend heavily on the quality of the COLMAP output. Poor camera registration produced fragmented point clouds, which directly affected downstream Gaussian Splatting techniques such as FastGS and SuGaR, reinforcing the importance of robust upstream preprocessing.
- Deploying the platform introduced practical system-level challenges involving GPU configuration, local networking, remote accessibility, and bandwidth limitations when serving large Gaussian Splat models over external tunnelling services, highlighting the importance of deployment planning alongside software development.

- The internship provided practical experience across the complete 3D reconstruction pipeline—from data acquisition and photogrammetry to web application development, deployment, and optimization—and demonstrated that improvements made in the early stages of the pipeline often have a greater impact on the final reconstruction than tuning downstream algorithms alone.

6. Tools & Technologies Used

1. Programming & Scripting - Python, JavaScript, TypeScript
2. Photogrammetry & 3D Reconstruction - COLMAP, FastGS, SuperSplat, SuGaR, MeshLab, Nerfstudio, Brush, RealityScan, Hugin, LixelStudio
3. Computer Vision & Point Cloud Processing - RANSAC, DBSCAN, OpenCV, FFmpeg
4. Web Development - React, Vite, Three.js, FastAPI
5. Deployment & Infrastructure - Nginx, SSH, LAN networking, ngrok, DuckDNS, Docker
6. Databases & Backend - PostgreSQL, Redis, Celery
7. Hardware - Insta360 360° Camera , ISFCR GPU Workstation (nvidia)

7. Future Scope

Scenix is functional end-to-end today, but several parts of the pipeline are deliberately first-pass implementations with clear room to grow. This section lays out what's planned next and why it matters, for anyone evaluating where the project is headed.

- **Richer, more reliable evidence (object) detection (Stage 8)** — Detection currently runs on a fixed set of text prompts through OwlViT. Planned work: a configurable prompt set so a user can tell the system what kinds of objects to look for per scene, and an upgrade path to stronger open-vocabulary detectors (Grounding DINO / SAM 2) for higher recall and tighter boxes. Each detected object would also carry a supporting-view count — how many camera angles independently confirmed it — as a built-in confidence signal. This is the core research contribution of the project and the area with the most active development.
- **A full evidence report (Stage 12)** — The report currently outputs a v1 PDF with room dimensions and a flat object table. The next version would attach a photo crop and supporting-view thumbnails to every object, include capture metadata (date, camera, number of panoramas, marker used), embed a render of the reconstructed scene, and offer a DOCX export alongside the PDF. The goal is a report that can stand on its own as a shareable summary of a scene, not just a table.
- **A measurable mesh, alongside the splat** — A Gaussian splat looks great but isn't a solid, closed surface — it can't easily tell you “This is a wall” for measurement purposes. The plan is a dual-output architecture: keep the splat for visual walkthroughs, and separately generate a proper mesh (via 2D Gaussian Splatting or classical multi-view stereo) to serve as the actual measurement surface. A direct SuGaR integration was tried and dropped — it conflicted with the FastGS build and the meshes it produced weren't good enough to justify keeping it.
- **Clickable objects in the final viewer (Stage 11)** — The in-browser splat viewer currently shows the raw reconstruction. Planned: clickable, labelled bounding boxes over each detected object directly in the 3D view, with toggle-able layers (e.g. show/hide all “furniture”, show/hide low-confidence detections), likely built on PlayCanvas or WebGPU for better performance at scale.
- **More flexible scale calibration** — Measurements currently rely on a single ArUco marker being visible in at least two views. Future versions would support multiple markers for cross-validation, and optionally a LiDAR reference for scenes where placing a printed marker isn't practical.
- **GPU workers that scale** — Right now the Celery worker runs on whichever machine it's launched on, with CUDA visibility handled by manually injected environment variables. The production target is Docker containers orchestrated by Kubernetes with the NVIDIA container runtime, so multiple reconstruction jobs can run in parallel across a GPU cluster instead of one machine.
- **Hardened preprocessing (Stage 3)** — Blur and duplicate filtering exist but are only partially tuned. Future work includes making the blur/duplicate thresholds configurable per capture device and surfacing rejected-frame reasons directly in the UI (the backend already logs them).

8. Research Direction

While **3D Gaussian Splatting** provides an efficient method for reconstructing photorealistic scenes, the primary research objective of **SCENIX** extends beyond reconstruction itself. The project aims to develop an intelligent 3D scene understanding framework by integrating semantic object detection, classification, localization, and measurement into the reconstruction pipeline. By combining photogrammetry, Gaussian Splatting, computer vision, and web-based visualization within a single automated workflow, the platform moves towards creating interactive and measurable digital twins rather than static 3D reconstructions.

An important direction explored during the internship was improving reconstruction quality through **adaptive preprocessing**. Instead of generating a fixed number of perspective images from every 360° panorama, future work will investigate dynamically generating additional cubemap views based on COLMAP registration performance. By increasing image overlap only in regions where reconstruction quality is poor, the system can improve camera registration, produce denser point clouds, and ultimately generate higher-quality Gaussian Splat reconstructions while maintaining computational efficiency.

9. Conclusion

This internship provided practical experience across the complete 3D reconstruction pipeline, from 360° image acquisition and photogrammetric reconstruction to Gaussian Splatting, web application development, deployment, and pipeline optimization. The outcome of the work is a functional end-to-end platform that automates the generation of interactive 3D scene reconstructions while providing visualization, preprocessing, and workflow management through a unified interface.

Beyond implementation, the internship highlighted the importance of robust preprocessing, accurate camera registration, and seamless integration of multiple reconstruction components in determining the quality of the final output. The work also established a strong foundation for future research in intelligent 3D scene understanding, with planned extensions towards automatic object localization, semantic scene analysis, adaptive reconstruction techniques, and advanced digital twin generation for forensic and other real-world applications.

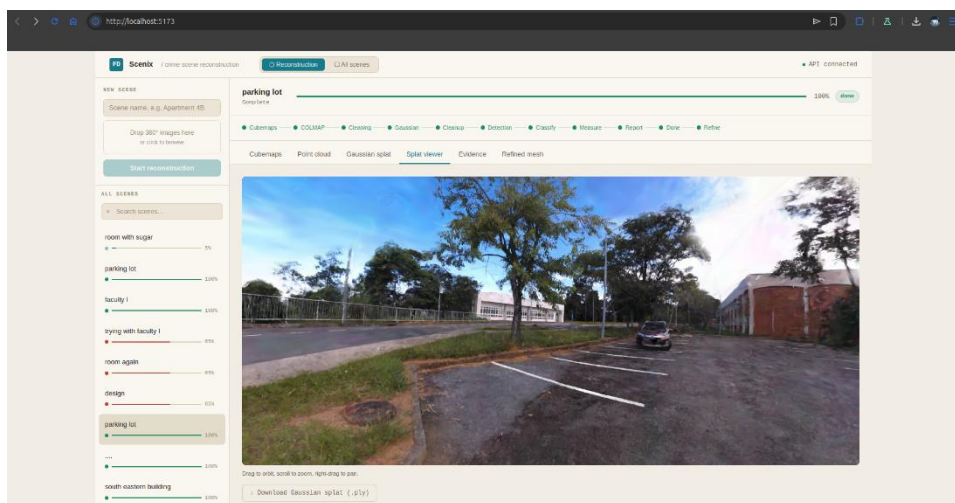


Figure 13 – Final UI pipeline

ACKNOWLEDGEMENT

I would like to express my gratitude to my guide Dr. Adithya Balasubramanyam, Professor, PES University for his continuous guidance, assistance and encouragement throughout the development of this project. I extend my acknowledgement to Mr. Manoj Kumar H R, Staff, PES University for the Co-Guidance throughout internship term.

I am grateful to the internship coordinator Prof Mahitha G and Dr Bhargavi M, Dept. of Computer Science and Engineering, PES University for organizing, managing and helping out with the entire process.

I take this opportunity to thank Dr. Shylaja S S, Chairperson, Department of Computer Science and Engineering & Dr. Jayashree Rangareddy, Chairperson, Department of CSE(AI&ML), PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof Jawahar Doreswamy, Chancellor, PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing me various opportunities and enlightenment every step of the way.

Finally, this internship could not have been completed without the continual support and encouragement I have received from my parents and my friends.